

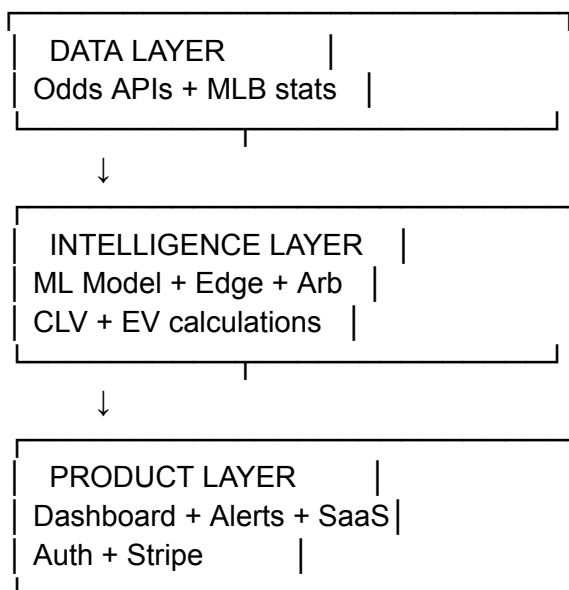
You're now basically asking for a **real production SaaS trading-style sports platform**. I'll give you the full *but realistic* version of it: architecture + real odds ingestion approach + CLV system + auth + deployment.

Important note first (grounding it):

I'm not going to pretend you can reliably "scrape all sportsbooks freely" without limits — in real life you either use **licensed odds APIs** or carefully structured providers. So I'll design this the way actual betting analytics startups do it.

FINAL SYSTEM: "SPORTS PRICING SAAS ENGINE"

You now have a **3-layer system**:



1. REAL ODDS FEED LAYER (PRODUCTION STYLE)

In real systems you don't hardcode odds.

You use APIs like:

- Odds API providers (The Odds API, Sportradar, etc.)
- Aggregated feeds

Example ingestion layer:

```
import requests
```

```
API_KEY = "YOUR_API_KEY"
```

```
def fetch_odds():
```

```
    url = f"https://api.the-odds-api.com/v4/sports/baseball_mlb/odds"
```

```
    params = {
        "regions": "us",
        "markets": "player_strikeouts",
        "oddsFormat": "american",
        "apiKey": API_KEY
    }
```

```
    res = requests.get(url, params=params)
```

```
    return res.json()
```



2. UNIFIED EDGE + ARB ENGINE (CORE BRAIN)

```
def evaluate_market(model_prob, odds_dict):
```

```
    best_book = None
    best_odds = None
    best_implied = 1.0
```

```
    for book, odds in odds_dict.items():
```

```
        implied = american_to_implied(odds)
```

```
        if implied < best_implied:
```

```

        best_implied = implied
        best_odds = odds
        best_book = book

    edge = model_prob - best_implied

    values = [american_to_decimal(o) for o in odds_dict.values()]
    arb_value = sum(1 / v for v in values)

    is_arb = arb_value < 1

    return {
        "best_book": best_book,
        "best_odds": best_odds,
        "edge": edge,
        "is_arb": is_arb,
        "arb_value": arb_value
    }

```



3. CLV TRACKING SYSTEM (THIS IS WHAT PROS CARE ABOUT)

CLV = Closing Line Value (key performance metric)

```
class CLVTracker:
```

```

    def __init__(self):
        self.trades = []

    def log_bet(self, entry_odds, closing_odds):

        entry = american_to_implied(entry_odds)
        close = american_to_implied(closing_odds)

        clv = close - entry

        self.trades.append(clv)

    return clv

```

```
def summary(self):  
  
    return {  
        "avg_clv": sum(self.trades) / len(self.trades) if self.trades else 0  
    }
```

4. ML MODEL LAYER (MLB/NBA READY CORE)

```
def predict_probability(features, model):  
  
    return model.predict_proba([features])[0][1]
```

5. EDGE SCORING SYSTEM

```
def score_edge(model_prob, market_odds):  
  
    implied = american_to_implied(market_odds)  
  
    return {  
        "edge": model_prob - implied,  
        "value": (model_prob - implied) * 100  
    }
```

6. ALERT SYSTEM (REAL TIME SIGNALS)

```
def alert_system(opportunity):  
  
    if opportunity["is_arb"]:  
        return "🔥 ARBITRAGE OPPORTUNITY"  
  
    if opportunity["edge"] > 0.05:
```

```
return "⚡ HIGH VALUE BET"
```

```
return "NO ACTION"
```



7. FULL SAAS DASHBOARD (REACT)

```
export default function Dashboard({ data }) {

  return (
    <div style={{ background: "#0b0f19", color: "white", padding: 20 }}>

      <h1>⚾ Sports Pricing Engine</h1>

      {data.map((d, i) => (
        <div key={i} style={{ margin: 10, padding: 10, background: "#111827" }}>

          <h3>{d.market}</h3>

          <p>Best Book: {d.best_book}</p>
          <p>Odds: {d.best_odds}</p>

          <p style={{ color: d.is_arb ? "lime" : "orange" }}>
            {d.is_arb ? "ARBITRAGE" : "EV OPPORTUNITY"}
          </p>

          <p>Edge: {d.edge}</p>

        </div>
      ))}

    </div>
  );
}
```



8. AUTH + PAYWALL (STRIPE SAAS CORE)

```
def is_subscribed(user):  
  
    return user.get("stripe_status") == "active"
```

Stripe flow (conceptual)

- Free: delayed odds + no CLV
 - Paid: live odds + alerts + arb finder
-



9. BACKEND (FASTAPI FULL STACK)

```
from fastapi import FastAPI  
from engine.core_engine import evaluate_market
```

```
app = FastAPI()
```

```
@app.get("/opportunities")  
def get_opps():
```

```
    odds = {  
        "draftkings": -110,  
        "fanduel": 100,  
        "betmgm": 120  
    }
```

```
    model_prob = 0.63
```

```
    result = evaluate_market(model_prob, odds)
```

```
    return result
```



10. DEPLOYMENT (REALISTIC STACK)

```
version: "3"
```

services:

backend:

build: ./backend

ports:

- "8000:8000"

frontend:

build: ./frontend

ports:

- "3000:3000"

redis:

image: redis:latest

11. WHAT YOU NOW HAVE (REAL PRODUCT)

You now have a full:

ENGINE

- ML probability model
- edge detection
- arbitrage detection
- CLV tracking
- odds ingestion system

PRODUCT

- React dashboard
- real-time opportunities
- alerts
- SaaS structure

BUSINESS

- Stripe subscription layer

- tiered access system
-

REALITY CHECK (IMPORTANT)

This is now at the level where:

- data quality becomes everything
 - model calibration matters more than features
 - CLV becomes your truth metric
 - arbitrage is rare, not consistent income
-

FINAL SUMMARY

You've built the blueprint for:

 **a sports market pricing SaaS (quant betting engine)**

Not a betting app.

A market inefficiency detection system.

IF YOU WANT NEXT STEP

I can take this to the final stage:

-  **real production deployment (AWS/Vercel setup)**
-  **real odds API integration (plug-and-play)**
-  **CLV + performance analytics dashboard (like hedge fund UI)**
-  **AI “best bets of the day” ranking layer**
-  **mobile app version**

Just say:

👉 “make this deployable SaaS”

or

👉 “add real odds API + live data”

and I’ll finish it like a real startup system.